

조합 논리 회로

Combination Logic Circuit;

임의의 시점에서의 출력 값이 그 시점의 입력 값에 의해서만 결정되는
논리회로;메모리(기억) 능력이 없음;

모든 조합 논리 회로를 부울대수식; 카르노맵 이용 최소화 하고
기본 디지털게이트 조합으로 표현; NAND로 구현 가능

03 조합 논리 회로

1 데이터 형태에 따른 분류

□ 조합 논리 회로의 개요

- 조합 논리 회로(combinational logic circuit)는 현재 입력 값으로 출력이 결정되는 회로

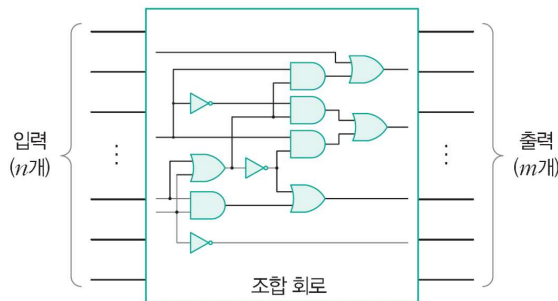


그림 3-32 조합 논리 회로의 개념도

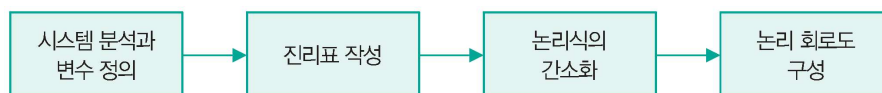


그림 3-33 조합 논리 회로의 설계 과정

03 조합 논리 회로

2 조합 논리 회로의 종류

□ 반가산기

- 반가산기(Half-Adder, HA)는 1자리 2진수 2개를 입력하여 합(S)과 캐리(Carry, C)를 출력하는 조합 논리 회로

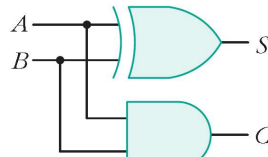
A	0	0	1	1
$+ B$	$+ 0$	$+ 1$	$+ 0$	$+ 1$
$\hline C \ S$	0 0	0 1	0 1	1 0

입력		출력	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

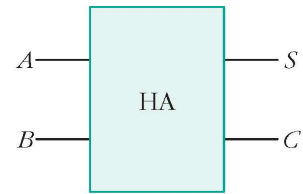
$$S = \bar{A}B + A\bar{B} = A \oplus B$$

$$C = AB$$

(a) 진리표와 논리식



(b) 논리 회로



(c) 논리 기호

그림 3-34 반가산기

03 조합 논리 회로

□ 전가산기

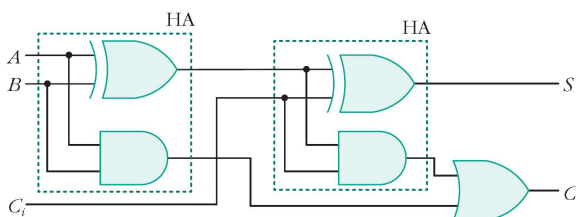
- 전가산기(Full-Adder, FA)는 2진수 입력 A , B 와 아랫자리에서 올라온 캐리 C_i 를 포함하여 1자리 2진수 3개를 더하는 조합 논리 회로

입력			출력	
A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

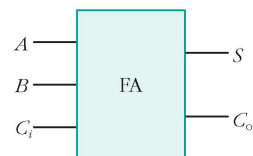
(a) 진리표

$$\begin{aligned}
 S &= \bar{A}\bar{B}C_i + \bar{A}B\bar{C}_i + A\bar{B}\bar{C}_i + ABC_i \\
 &= \bar{A}(\bar{B}C_i + B\bar{C}_i) + A(\bar{B}\bar{C}_i + BC_i) \\
 &= \bar{A}(B \oplus C_i) + A(\overline{B \oplus C_i}) \\
 &= A \oplus (B \oplus C_i) = (A \oplus B) \oplus C_i \\
 C_o &= \bar{A}BC_i + \bar{A}B\bar{C}_i + A\bar{B}\bar{C}_i + ABC_i \\
 &= C_i(\bar{A}B + A\bar{B}) + AB(\bar{C}_i + C_i) \\
 &= C_i(A \oplus B) + AB
 \end{aligned}$$

(b) 논리식의 간소화



(c) 논리 회로



(d) 논리 기호

그림 3-35 전가산기

03 조합 논리 회로

입력			출력	
A	B	C _i	S	C _o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(a) 진리표

$$\begin{aligned}
 S &= \overline{A}\overline{B}C_i + \overline{A}B\overline{C}_i + A\overline{B}\overline{C}_i + ABC_i \\
 &= \overline{A}(\overline{B}C_i + B\overline{C}_i) + A(\overline{B}\overline{C}_i + BC_i) \\
 &= \overline{A}(B \oplus C_i) + A(\overline{B} \oplus \overline{C}_i) \\
 &= A \oplus (B \oplus C_i) = (A \oplus B) \oplus C_i \\
 C_o &= \overline{A}BC_i + \overline{A}B\overline{C}_i + A\overline{B}\overline{C}_i + ABC_i \\
 &= C_i(\overline{A}B + A\overline{B}) + AB(\overline{C}_i + C_i) \\
 &= C_i(A \oplus B) + AB
 \end{aligned}$$

(b) 논리식의 간소화

입력		출력
A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

(a) 2입력인 경우: $F = \overline{A}B + A\overline{B} = A \oplus B$
그림 3-9 XOR 게이트의 진리표와 논리 기호

입력		출력
A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

(a) 2입력인 경우: $F = \overline{A \oplus B} = A \odot B$
그림 3-10 XNOR 게이트의 진리표와 논리 기호

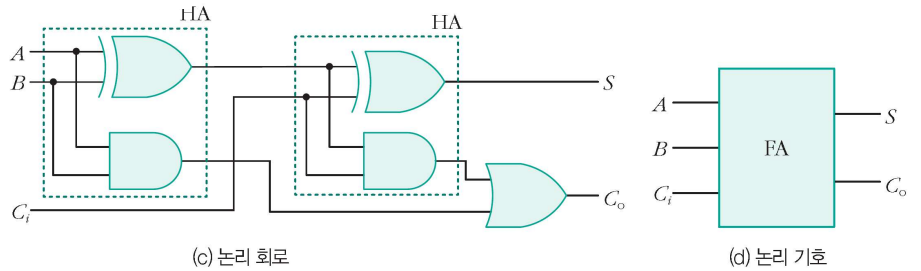


그림 3-35 전가산기

03 조합 논리 회로

□ 반감산기

- 반감산기(Half-Subtractor, HS)는 1비트 2진수 A에서 B를 빼 차(D)와 빌림 수(K)를 계산하는 뺄셈회로

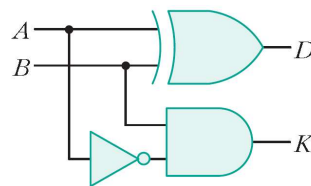
입력		출력		설명(D=A-B)		
A	B	D	K	뺄 수 없으면 윗자리에서 빌려 와 계산한다.		
0	0	0	0	0-0=0	빌림 수 없음 (K=0)	∴ D=0
0	1	1	1	0-1=-1	빌림 수 2 (K=1)	∴ D=2-1=1
1	0	1	0	1-0=1	빌림 수 없음 (K=0)	∴ D=1
1	1	0	0	1-1=0	빌림 수 없음 (K=0)	∴ D=0

(a) 진리표와 원리

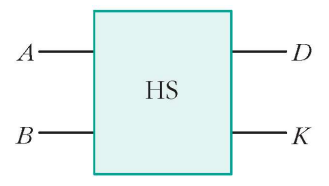
$$D = \overline{A}B + A\overline{B} = A \oplus B$$

$$K = \overline{A}B$$

(b) 논리식



(c) 논리 회로



(d) 논리 기호

그림 3-36 반감산기

03 조합 논리 회로

전감산기

- 전감산기(Full-Subtractor, FS)는 2진수 입력 A , B 와 아랫자리로 빌려주는 수 K_i 를 포함하여 $A-B-K_i$ 를 계산하는 조합 논리 회로

입력			출력		설명($D=A-B-K_i$) 빌 수 없으면 윗자리에서 빌려 와 계산한다.
A	B	K_i	D	K_o	
0	0	0	0	0	$0-0-0=0$, 빌림 수 없음($K_o=0$) $\therefore D=0$
0	0	1	1	1	$0-0-1=-1$, 빌림 수 2($K_o=1$) $\therefore D=2-1=1$
0	1	0	1	1	$0-1-0=-1$, 빌림 수 2($K_o=1$) $\therefore D=2-1=1$
0	1	1	0	1	$0-1-1=-2$, 빌림 수 2($K_o=1$) $\therefore D=2-2=0$
1	0	0	1	0	$1-0-0=1$, 빌림 수 없음($K_o=0$) $\therefore D=1$
1	0	1	0	0	$1-0-1=0$, 빌림 수 없음($K_o=0$) $\therefore D=0$
1	1	0	0	0	$1-1-0=0$, 빌림 수 없음($K_o=0$) $\therefore D=0$
1	1	1	1	1	$1-1-1=-1$, 빌림 수 2($K_o=1$) $\therefore D=2-1=1$

입력	출력	$F = \bar{A}\bar{B} + AB$
A	B	
0	0	1
0	1	0
1	0	0
1	1	1

(a) 2입력인 경우: $F = \bar{A} \oplus \bar{B} = A \odot B$

그림 3-10 XNOR 게이트의 진리표와 논리 기호

(a) 진리표와 원리

입력	출력
A	F
0	0
0	1
1	1
1	0

(a) 2입력인 경우: $F = \bar{A}B + A\bar{B} = A \oplus B$

그림 3-9 XOR 게이트의 진리표와 논리 기호

BK_i	00	01	11	10
A	0	1	1	1
1	1	1	1	1

$D = A \oplus B \oplus K_i$

BK_i	00	01	11	10
A	0	1	1	1
1	1	1	1	1

$K_o = \bar{A}B + \bar{A}K_i + BK_i$

(b) 논리식의 유도

$$\begin{aligned}
 K_o &= \bar{A}\bar{B}K_i + \bar{A}BK_i + \bar{A}\bar{B}\bar{K}_i + \bar{A}B\bar{K}_i \\
 &= (\bar{A}\bar{B} + \bar{A}B)K_i + \bar{A}B(K_i + \bar{K}_i) \\
 &= (\bar{A} \oplus B)K_i + \bar{A}B
 \end{aligned}$$

(c) 논리식의 간소화

03 조합 논리 회로

전감산기(계속)

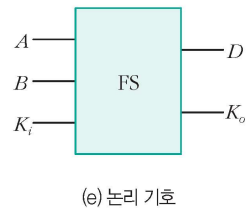
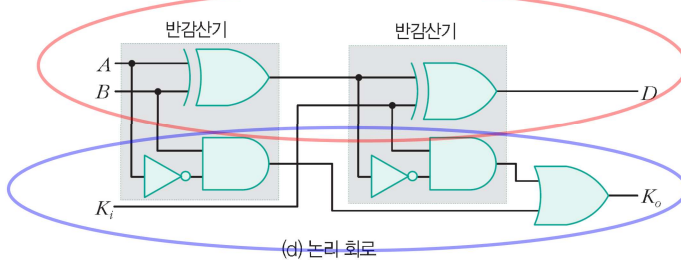


그림 3-37 전감산기

BK_i	00	01	11	10
A	0	1	1	1
1	1	1	1	1

$D = A \oplus B \oplus K_i$

BK_i	00	01	11	10
A	0	1	1	1
1	1	1	1	1

$K_o = \bar{A}B + \bar{A}K_i + BK_i$

(b) 논리식의 유도

$$\begin{aligned}
 K_o &= \bar{A}\bar{B}K_i + \bar{A}BK_i + \bar{A}\bar{B}\bar{K}_i + \bar{A}B\bar{K}_i \\
 &= (\bar{A}\bar{B} + \bar{A}B)K_i + \bar{A}B(K_i + \bar{K}_i) \\
 &= (\bar{A} \oplus B)K_i + \bar{A}B
 \end{aligned}$$

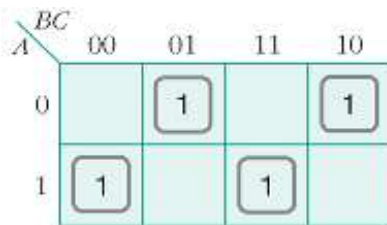
(c) 논리식의 간소화

진리표 $\leftrightarrow$ 부울대수식(or 카르노맵) $\leftrightarrow$ 조합논리회로

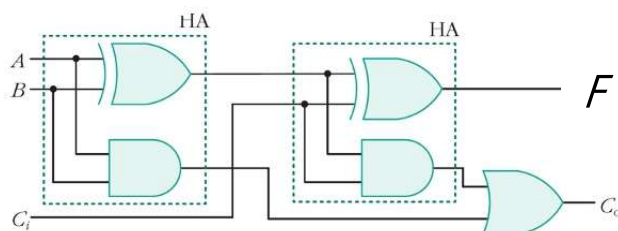
입력			출력
A	B	C _i	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(a) 진리표

$$\begin{aligned}
 F &= \overline{A}\overline{B}C_i + \overline{A}B\overline{C}_i + A\overline{B}\overline{C}_i + ABC_i \\
 &= \overline{A}(\overline{B}C_i + B\overline{C}_i) + A(\overline{B}\overline{C}_i + BC_i) \\
 &= \overline{A}(B \oplus C_i) + A(\overline{B} \oplus \overline{C}_i) \\
 &= A \oplus (B \oplus C_i) = (A \oplus B) \oplus C_i
 \end{aligned}$$



(a) $F = A \oplus B \oplus C_i$



```

Users > hertzkim > Dropbox (Personal) > 강의 > 2022-02 > 컴퓨터
1  #include <stdio.h>
2
3  int digital_logic(int A, int B, int C)
4  {
5      int F;
6      F = (A ^ B) ^ C;
7      return F;
8  }
9
10 int main(void)
11 {
12     printf("F = %d\n", digital_logic(0,0,0));
13     printf("F = %d\n", digital_logic(0,0,1));
14     printf("F = %d\n", digital_logic(0,1,0));
15     printf("F = %d\n", digital_logic(0,1,1));
16     printf("F = %d\n", digital_logic(1,0,0));
17     printf("F = %d\n", digital_logic(1,0,1));
18     printf("F = %d\n", digital_logic(1,1,0));
19     printf("F = %d\n", digital_logic(1,1,1));
20 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

Hyunseoks-MacBook-Pro:코드 작업 hertzkim\$ cd "/Users/he
 조 /코드 작업 /"digital_logic
 F = 0
 F = 1
 F = 1
 F = 0
 F = 1
 F = 0
 F = 1
 F = 0
 F = 1

03 조합 논리 회로

병렬 가감산기

병렬 가산기(parallel-adder) : 전가산기 여러 개를 병렬로 연결하여 만든 2비트 이상의 가산기

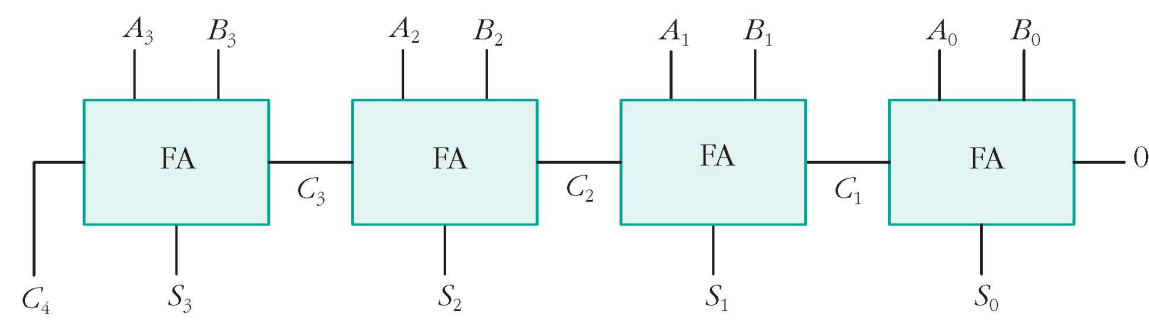


그림 3-38 전가산기를 이용한 병렬 가산기

입력			출력	
A	B	C _i	S	C _o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

A: A₃A₂A₁A₀
+ B: B₃B₂B₁B₀

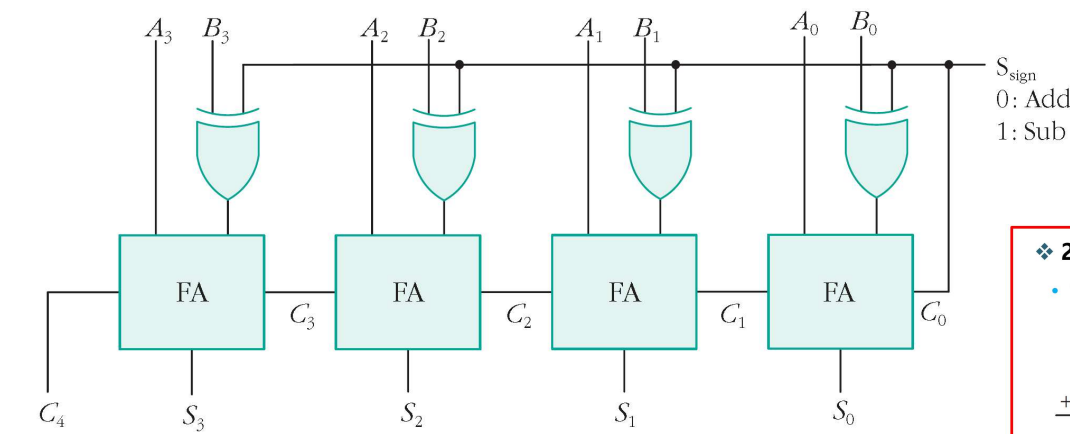
C₄C₃C₂C₁x
S₃S₂S₁S₀

A: 1101
+ B: 0101

C:11010
S: 0010

03 조합 논리 회로

병렬 가감산기(parallel-adder/subtractor) : 병렬 가산기의 B입력을 부호 S(sign)와 XOR하여 전가산기의 입력으로 사용함으로써 덧셈과 뺄셈이 모두 가능한 회로



입력	출력
S	F
0	0
0	1
1	1
1	0

(a) 2입력인 경우: $F = \overline{A}B + A\overline{B} = A \oplus B$

그림 3-39 병렬 가감산기

2의 보수 방식

1의 보수 + 1 = 2의 보수

0 0 0 0 0 0 1 1
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
1 1 1 1 1 1 0 0 ← 1의 보수
+
1 1 1 1 1 1 0 1 ← 2의 보수

전가산기 여러 개를 병렬로 연결하여 만든 2비트 이상의 가산기를 병렬 가산기라고 함.
B입력을 부호 S와 XOR 하면 전가산기를 덧셈 뺄셈 모두 가능한 **병렬 가감산기**가 됨.

S=0이면 B 값이 전가산기로 그대로 입력되어 덧셈이 되고,
S=1이면 B 값이 반전 되어 **1의 보수가 됨**. 이때 **C₀ = 1**이 되기 때문에 B의 1의 보수에 1이 더해져 **2의 보수가 됨**

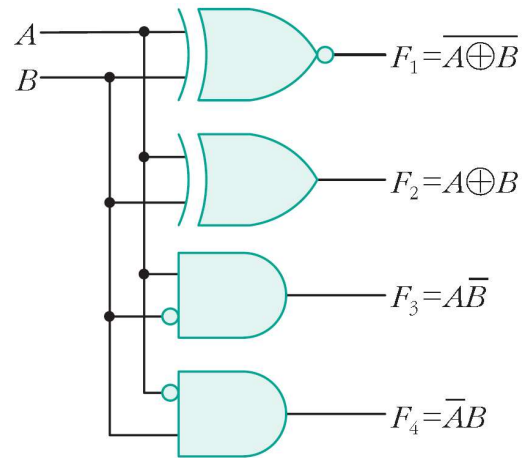
03 조합 논리 회로

□ 비교기

- **2진 비교기**(comparator)는 두 2진수 값의 크기를 비교하는 회로다.

입력		출력			
A	B	$A=B$ F_1	$A \neq B$ F_2	$A > B$ F_3	$A < B$ F_4
0	0	1	0	0	0
0	1	0	1	0	1
1	0	0	1	1	0
1	1	1	0	0	0

(a) 진리표



(b) 논리 회로

그림 3-40 1비트 비교기

03 조합 논리 회로

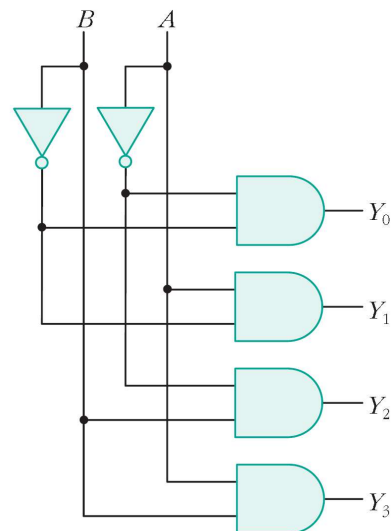
□ 디코더

- **디코더**(decoder) : 입력선에 나타나는 n 비트의 2진 코드를 최대 2^n 개의 서로 다른 정보로 바꿔주는 조합논리회로

입력		출력			
B	A	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

$$Y_0 = \overline{B}\overline{A}, Y_1 = \overline{B}A, Y_2 = B\overline{A}, Y_3 = BA$$

(a) 진리표와 논리식



(b) 논리 회로

그림 3-41 2×4 디코더

03 조합 논리 회로

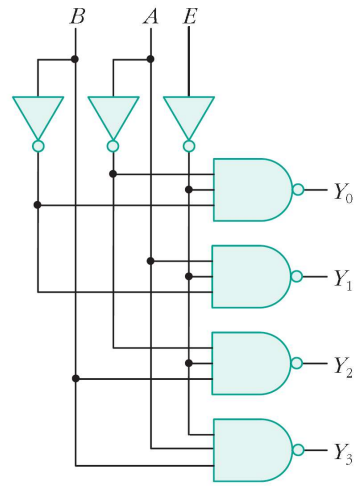
❖ 실제 디코더 회로

- 실제 IC들은 AND 게이트가 아닌 NAND 게이트로 구성되어 출력은 그림과 같이 반대로 된다.
- 또 대부분의 디코더 IC는 인에이블(enable) 입력이 있어 회로를 제어한다.

입력			출력			
$\bar{E}$	B	A	Y_3	Y_2	Y_1	Y_0
1	x	x	1	1	1	1
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1

$$Y_0 = \overline{EBA}, Y_1 = \overline{E\bar{B}A}, Y_2 = \overline{EBA}, Y_3 = \overline{E\bar{B}A}$$

(a) 진리표와 논리식



(b) 논리 회로

그림 3-42 인에이블이 있는 2×4 NAND 디코더

03 조합 논리 회로

❖ 3×8 디코더

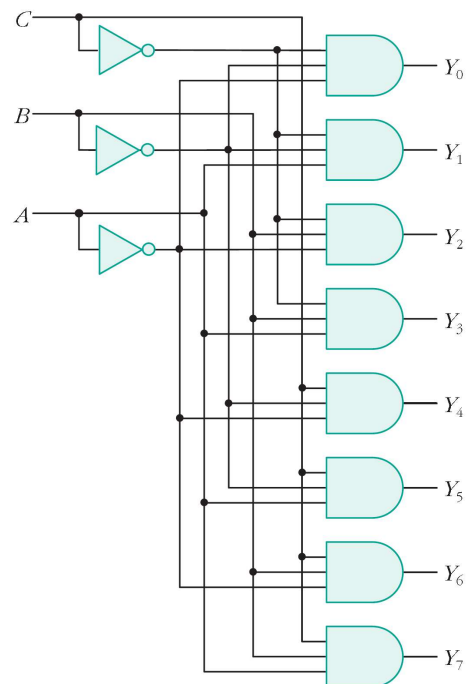
- 3개의 입력에 따라서 8개의 출력 중 하나가 선택

입력			출력							
C	B	A	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

$$Y_0 = \overline{CBA}, Y_1 = \overline{C\bar{B}A}, Y_2 = \overline{CBA}, Y_3 = \overline{C\bar{B}A}$$

$$Y_4 = \overline{CBA}, Y_5 = \overline{C\bar{B}A}, Y_6 = \overline{CBA}, Y_7 = \overline{CBA}$$

(a) 진리표와 논리식



(b) 논리 회로

그림 3-43 3×8 디코더 회로

03 조합 논리 회로

❖ 2개의 3×8 디코더로 4×16 디코더를 구성

$D=0$	상위 디코더만 enable되어 출력은 $Y_0 \sim Y_7$ 중의 하나가 1로 되고, 하위 디코더 출력들은 모두 0이 된다.
$D=1$	하위 디코더만 enable 되어 출력은 $Y_8 \sim Y_{15}$ 중의 하나가 1로 되고, 상위 디코더 출력들은 모두 0이 된다.

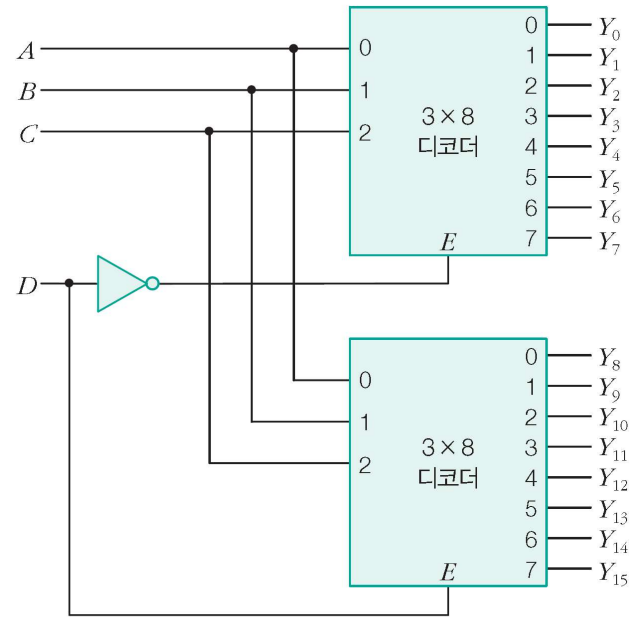


그림 3-44 3×8 디코더 2개를 이용한 4×16 디코더

03 조합 논리 회로

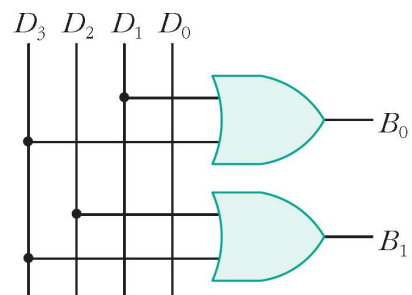
□ 인코더

- 부호기라고도 하는 인코더(encoder)는 디코더의 반대 기능을 수행하는 조합 논리 회로로, 2^n 개를 입력받아 n 개를 출력한다.
- 인코더는 2^n 개 중 활성화된 1비트 입력 신호를 받아 그 숫자에 해당하는 n 비트 2진 정보를 출력한다.

입력				출력	
D_3	D_2	D_1	D_0	B_1	B_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

$$B_1 = D_2 + D_3, B_0 = D_1 + D_3$$

(a) 진리표와 논리식



(b) 논리 회로

그림 3-45 4×2 인코더

03 조합 논리 회로

❖ 8×3 인코더

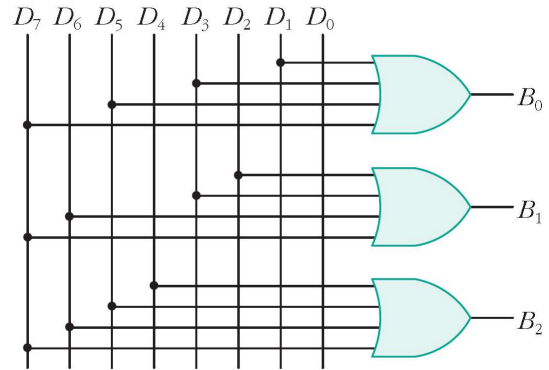
- $8(=2^3)$ 개의 입력과 3개의 출력을 가지며, 입력의 신호에 따라 3개의 2진 조합으로 출력한다.

입력								출력		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	B_2	B_1	B_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$B_2 = D_4 + D_5 + D_6 + D_7, \quad B_1 = D_2 + D_3 + D_6 + D_7$$

$$B_0 = D_1 + D_3 + D_5 + D_7$$

(a) 진리표와 논리식



(b) 논리 회로

그림 3-46 8×3 인코더

03 조합 논리 회로

□ 멀티플렉서

- **멀티플렉서**(multiplexer, MUX)는 여러 개의 입력선들 중에서 하나를 선택하여 출력선에 연결하는 조합논리회로이다. 선택선들의 값에 따라서 특별한 입력선이 선택된다.
- 멀티플렉서는 많은 입력들 중 하나를 선택하여 선택된 입력선의 2진 정보를 출력선에 넘겨주기 때문에 **데이터 선택기**(data selector)라 부르기도 한다.
- **디멀티플렉서**(demultiplexer, DEMUX)는 정보를 한 선으로 받아 2^n 개의 가능한 출력 선들 중 하나를 선택하여, 받은 정보를 전송하는 회로다.

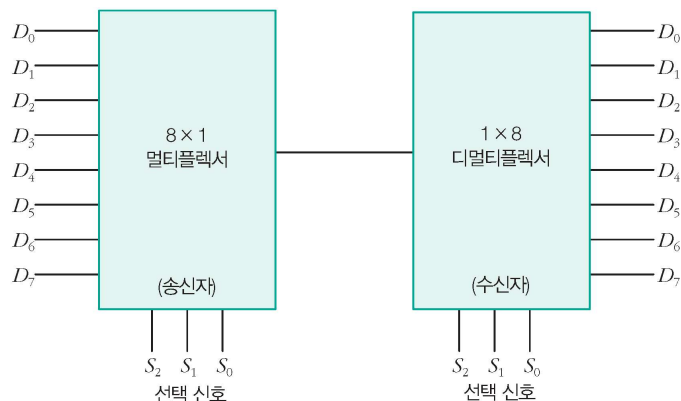


그림 3-47 멀티플렉서와 디멀티플렉서의 역할

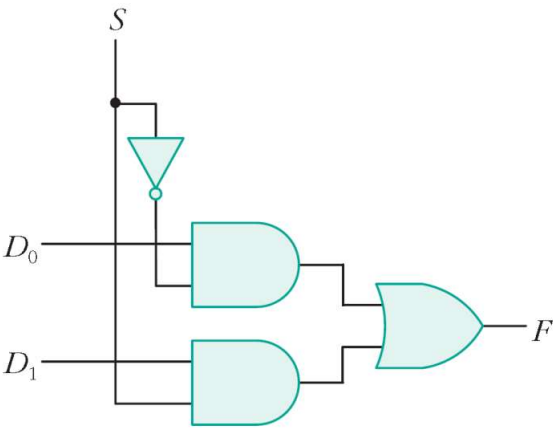
❖ 2×1 멀티플렉서

- $2(=2^1)$ 개의 입력중의 하나를 선택선 S 에 입력된 값에 따라서 출력으로 보내주는 조합회로

선택선 S	출력 F
0	D_0
1	D_1

$$F = \bar{S}D_0 + SD_1$$

(a) 진리표와 논리식



(b) 논리 회로

그림 3-48 2×1 멀티플렉서

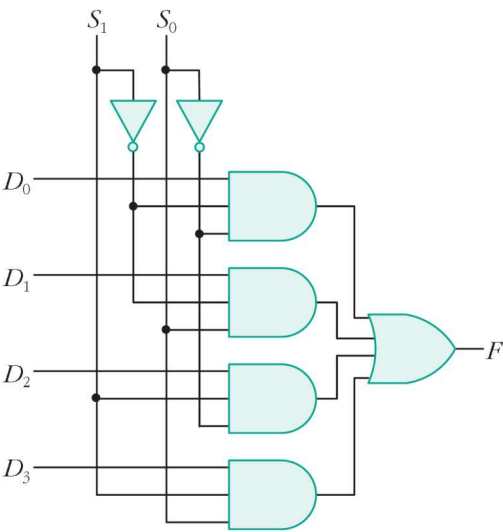
❖ 4×1 멀티플렉서

- $4(=2^2)$ 개의 입력중의 하나를 선택선 S_1 과 S_0 에 입력된 값에 따라서 출력으로 보내주는 조합회로

선택선		출력 F
S_1	S_0	
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

$$F = \bar{S}_1\bar{S}_0D_0 + \bar{S}_1S_0D_1 + S_1\bar{S}_0D_2 + S_1S_0D_3$$

(a) 진리표와 논리식



(b) 논리 회로

그림 3-49 4×1 멀티플렉서

03 조합 논리 회로

❖ 4×1 멀티플렉서 응용

- 4×1 멀티플렉서를 이용하여 두 입력 A, B 에 대해 AND, OR, XOR, NOT 논리 연산을 수행하는 하드웨어 모듈

S_1, S_0 이 컴퓨터 명령어라면, $[S_1 S_0]$ 값에 따라서 AND, OR, XOR, NOT 같은 연산을 실행할 수 있음

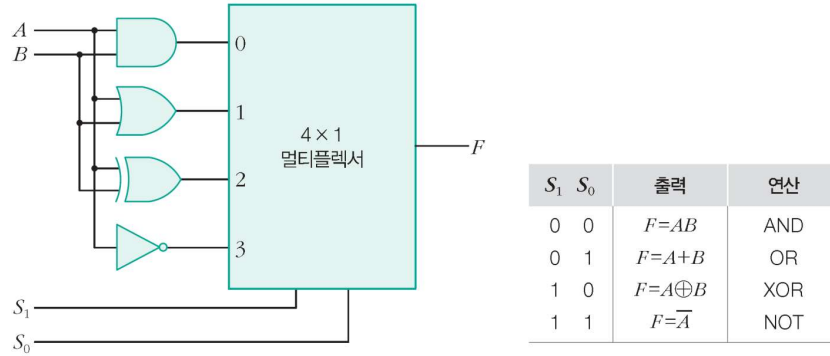


그림 3-50 4×1 멀티플렉서를 이용한 논리 연산 하드웨어 모듈

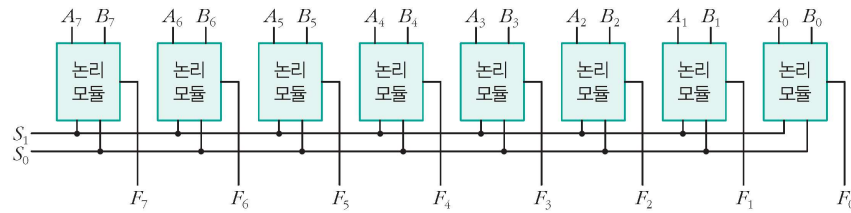


그림 3-51 8비트 논리 연산 장치 구성 예

03 조합 논리 회로

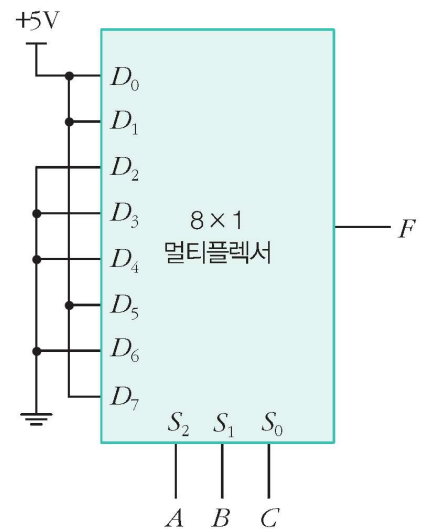
❖ 멀티플렉서를 이용한 조합회로 구현

- $F(A, B, C) = \sum m(0, 1, 5, 7)$ 를 8×1 멀티플렉서로 구현하는 경우
3개의 선택선을 입력 A, B, C 로 사용

입력			출력
A	B	C	F
0	0	0	1(D_0)
0	0	1	1(D_1)
0	1	0	0(D_2)
0	1	1	0(D_3)
1	0	0	0(D_4)
1	0	1	1(D_5)
1	1	0	0(D_6)
1	1	1	1(D_7)

$$F(A, B, C) = \sum m(0, 1, 5, 7)$$

(a) 진리표와 논리식



(b) 회로도

그림 3-52 8×1 멀티플렉서를 이용한 회로

03 조합 논리 회로

❖ 멀티플렉서를 이용한 조합회로 구현(계속)

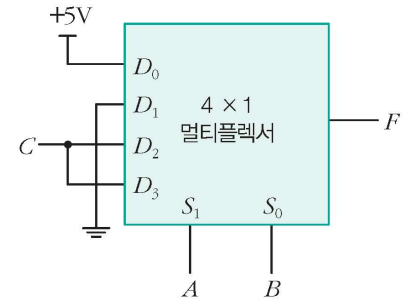
- $F(A, B, C) = \sum m(0, 1, 5, 7)$ 를 4×1 멀티플렉서로 구현하는 경우

☞ A, B 는 선택선으로 C 는 D_0, D_1, D_2, D_3 을 조합하여 사용

입력			출력	
A	B	C	F	
0	0	0	$D_0 = 1$	1
		1		1
0	1	0	$D_1 = 0$	0
		1		0
1	0	0	$D_2 = C$	0
		1		1
1	1	0	$D_3 = C$	0
		1		1

$$F(A, B, C) = \sum m(0, 1, 5, 7)$$

(a) 진리표와 논리식



(b) 회로도

그림 3-53 4×1 멀티플렉서를 이용한 회로

03 조합 논리 회로

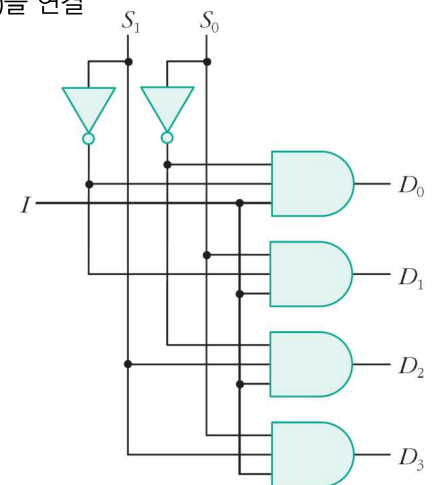
□ 디멀티플렉서

- **디멀티플렉서**(demultiplexer)는 하나의 입력선에 데이터를 입력하면 선택선 n 개로 2^n 개 중 하나를 출력하는 조합 논리 회로다. **데이터 분배기**라고도 한다.
- 1×4 디멀티플렉서는 선택선(S_1, S_0) 2개로 출력(D_3, D_2, D_1, D_0) 4개 중 하나를 선택해 입력(I)을 연결한다.

선택선		출력			
S_1	S_0	D_3	D_2	D_1	D_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

$$D_0 = \bar{S}_1 \bar{S}_0 I, D_1 = \bar{S}_1 S_0 I, D_2 = S_1 \bar{S}_0 I, D_3 = S_1 S_0 I$$

(a) 진리표와 논리식



(b) 논리 회로

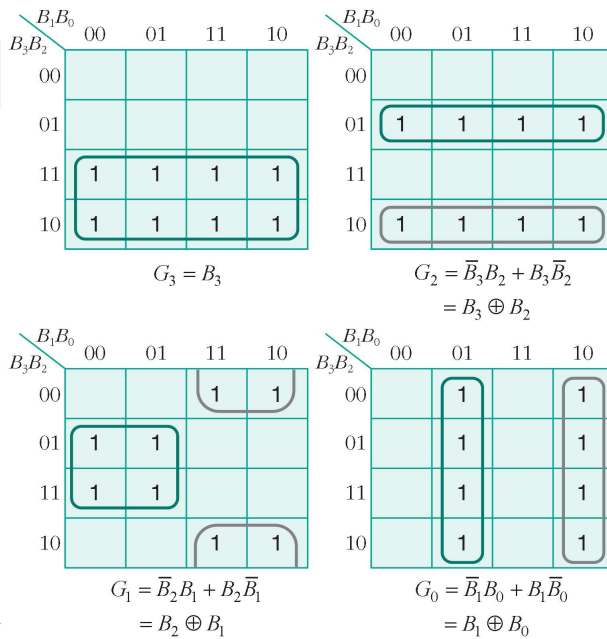
그림 3-54 1×4 디멀티플렉서

03 조합 논리 회로

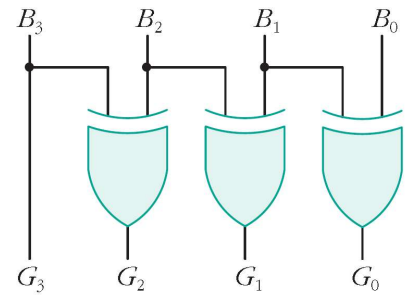
□ 코드 변환기 (2진 코드 → 그레이 코드 변환)

2진 코드(입력) $B_3 B_2 B_1 B_0$	그레이 코드(출력) $G_3 G_2 G_1 G_0$
0 0 0 0	0 0 0 0
0 0 0 1	0 0 0 1
0 0 1 0	0 0 1 1
0 0 1 1	0 0 1 0
0 1 0 0	0 1 1 0
0 1 0 1	0 1 1 1
0 1 1 0	0 1 0 1
0 1 1 1	0 1 0 0
1 0 0 0	1 1 0 0
1 0 0 1	1 1 0 1
1 0 1 0	1 1 1 1
1 0 1 1	1 1 1 0
1 1 0 0	1 0 1 0
1 1 0 1	1 0 1 1
1 1 1 0	1 0 0 1
1 1 1 1	1 0 0 0

(a) 진리표



(b) 논리식의 간소화



(c) 논리 회로

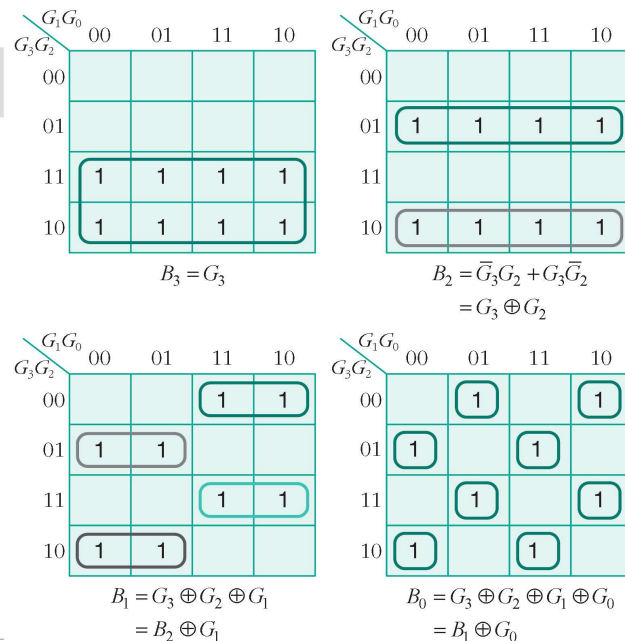
그림 3-55 2진 코드를 그레이 코드로 변환하는 회로

03 조합 논리 회로

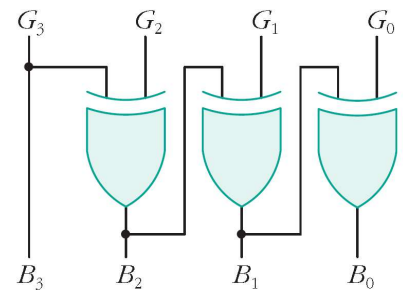
□ 코드 변환기 (그레이 코드 → 2진 코드 변환)

그레이 코드(입력) $G_3 G_2 G_1 G_0$	2진 코드(출력) $B_3 B_2 B_1 B_0$
0 0 0 0	0 0 0 0
0 0 0 1	0 0 0 1
0 0 1 0	0 0 1 1
0 0 1 1	0 0 1 0
0 1 0 0	0 1 1 1
0 1 0 1	0 1 1 0
0 1 1 0	0 1 0 0
0 1 1 1	0 1 0 1
1 0 0 0	1 1 1 1
1 0 0 1	1 1 1 0
1 0 1 0	1 1 0 0
1 0 1 1	1 1 0 1
1 1 0 0	1 0 0 0
1 1 0 1	1 0 0 1
1 1 1 0	1 0 1 1
1 1 1 1	1 0 1 0

(a) 진리표



(b) 논리식의 간소화



(c) 논리 회로

그림 3-56 그레이 코드를 2진 코드로 변환하는 회로

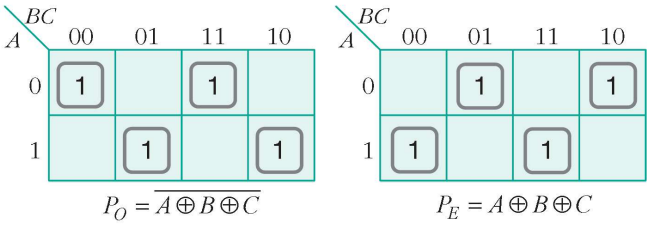
03 조합 논리 회로

패리티 발생기와 검사기

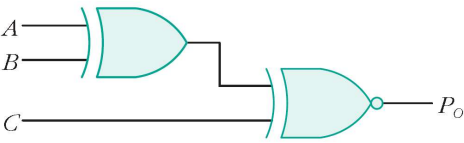
❖ 패리티 발생기

입력			출력	
A	B	C	P _O (홀수)	P _E (짝수)
0	0	0	1	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1

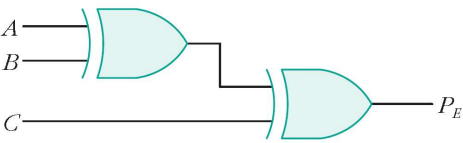
(a) 진리표



(b) 논리식의 간소화



(c) 홀수 패리티 발생기 논리 회로



(d) 짝수 패리티 발생기 논리 회로

그림 3-58 패리티 발생기

03 조합 논리 회로

❖ 패리티 검사기

Y=0 이면 NO ERROR

입력			출력	
A	B	C	P _O (홀수)	Y _O (홀수)
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

입력			출력	
A	B	C	P _E (짝수)	Y _E (짝수)
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

(a) 진리표

03 조합 논리 회로

❖ 패리티 검사기(계속)

- 패리티 검사기 출력이 $Y = 0$ 이면 에러가 발생하지 않았다고 판단하고, $Y = 1$ 이면 에러가 발생했다고 판단한다.

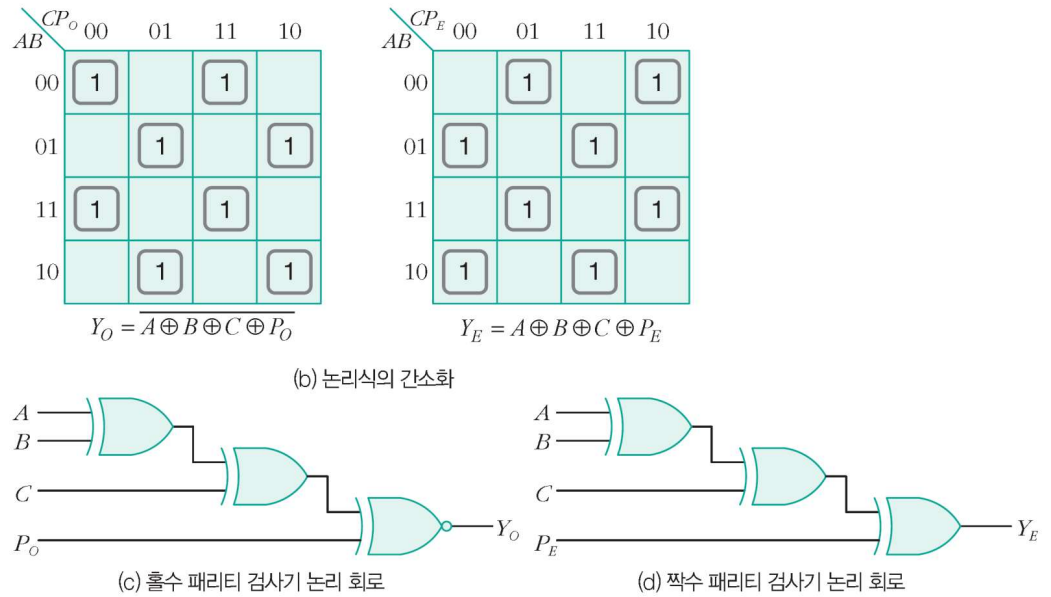


그림 3-59 패리티 검사기

03 조합 논리 회로

❖ 데이터 전송 시스템

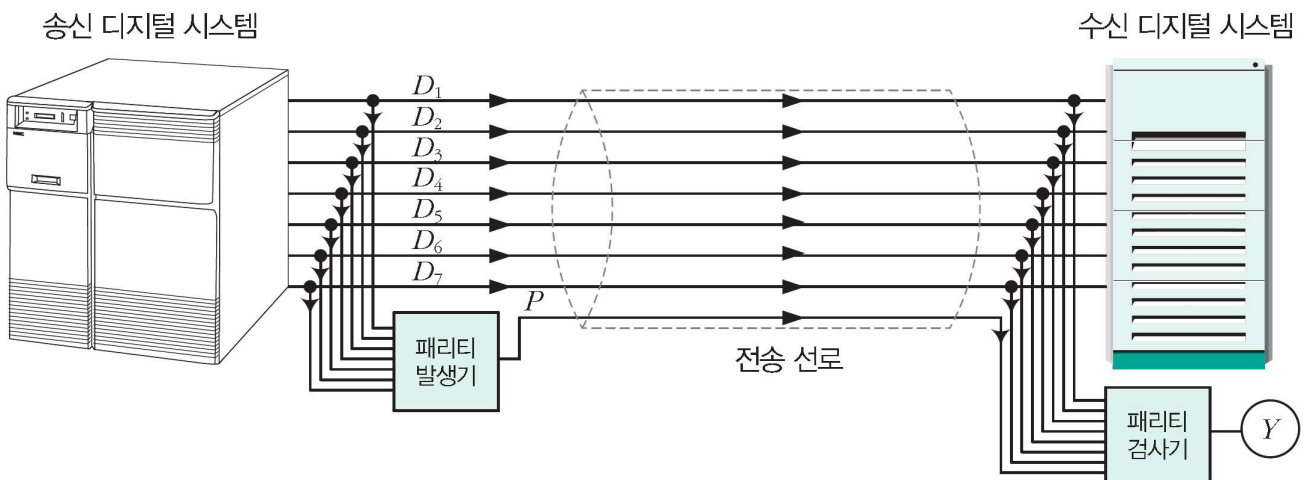


그림 3-60 데이터 전송 시스템에서 패리티 비트를 이용한 에러 검출